


# Computer Networks

# Redes de Computadores

## RC

## Teórica DOIS

## LEI

João Leitão

©2026 NOVA FCT

# Aula Dois

- Abstrair a Complexidade da Rede
- A abordagem em pilha e o Modelo OSI
- Como interagir com a pilha de rede: Sockets

# Redes de Computadores

- Da última vez, terminámos com uma ideia: a Internet é uma rede de redes que **se coordenam para oferecer um serviço comum**.
- Mas cada uma dessas redes pode ser completamente diferente por baixo — Wi-Fi aqui, fibra ali, talvez uma ligação por satélite algures.
- Por isso, a primeira pergunta de hoje é: como é que isto sequer funciona?

# A Necessidade de Criar Abstrações

- Imaginem que tinham de implementar um browser Web.

# A Necessidade de Criar Abstrações

- Imaginem que tinham de implementar um browser Web.
- E que, para o pôr a falar com a rede, tinham pessoalmente de lidar com Wi-Fi, Ethernet, fibra, ou satélite — o que quer que estivesse por baixo.

# A Necessidade de Criar Abstrações

- Imaginem que tinham de implementar um browser Web.
- E que, para o pôr a falar com a rede, tinham pessoalmente de lidar com Wi-Fi, Ethernet, fibra, ou satélite — o que quer que estivesse por baixo.
- Já agora — o que acontece quando surge uma nova tecnologia de ligação no próximo ano? Reescrevem o browser?

# A Necessidade de Criar Abstrações

- Imaginem que tinham de implementar um browser Web.
- E que, para o pôr a falar com a rede, tinham pessoalmente de lidar com Wi-Fi, Ethernet, fibra, ou satélite — o que quer que estivesse por baixo.
- Já agora — o que acontece quando surge uma nova tecnologia de ligação no próximo ano? Reescrevem o browser?
- E se o mesmo browser também tiver de funcionar através da ligação móvel do vosso telemóvel, ao mesmo tempo?

# A Necessidade de Criar Abstrações

- Imaginem que tinham de implementar um browser Web.
- E que, para o pôr a falar com a rede, tinham pessoalmente de lidar com Wi-Fi, Ethernet, fibra, ou satélite — o que quer que estivesse por baixo.
- Já agora — o que acontece quando surge uma nova tecnologia de ligação no próximo ano? Reescrevem o browser?
- E se o mesmo browser também tiver de funcionar através da ligação móvel do vosso telemóvel, ao mesmo tempo?
- Isto claramente não é como funciona na prática — então o que é que torna isto possível?

# Usar Camadas de Abstração

Conclusão: dividir o problema em **camadas**, cada uma oferece um serviço bem definido à(s) camada(s) acima, sem que essa camada precise de saber como o serviço abaixo é na verdade implementado.

# Nó, Ligação, Protocolo

- Está na hora de formalizar o vocabulário da última aula — já conheceram os três, informalmente, como “servidores”, “routers”, “switches”, “um gateway”.

# Nó, Ligação, Protocolo

- Está na hora de formalizar o vocabulário da última aula — já conheceram os três, informalmente, como “servidores”, “routers”, “switches”, “um gateway”.
- **Host/Nó**: qualquer dispositivo ligado à rede — um host terminal, ou um elemento de comutação/encaminhamento como um switch ou um router.

# Nó, Ligação, Protocolo

- Está na hora de formalizar o vocabulário da última aula — já conheceram os três, informalmente, como “servidores”, “routers”, “switches”, “um gateway”.
- **Host/Nó**: qualquer dispositivo ligado à rede — um host terminal, ou um elemento de comutação/encaminhamento como um switch ou um router.
- **Ligação**: o canal físico ou lógico que liga diretamente dois nós.

# Nó, Ligação, Protocolo

- Está na hora de formalizar o vocabulário da última aula — já conheceram os três, informalmente, como “servidores”, “routers”, “switches”, “um gateway”.
- **Host/Nó**: qualquer dispositivo ligado à rede — um host terminal, ou um elemento de comutação/encaminhamento como um switch ou um router.
- **Ligação**: o canal físico ou lógico que liga diretamente dois nós.
- **Protocolo**: o acordo entre os dois lados de uma camada sobre o formato exato das mensagens que trocam, e sobre o que cada lado faz ao enviar ou receber uma.

# Nó, Ligação, Protocolo

Conclusão: um protocolo tem sempre duas metades — a sintaxe da mensagem, e o comportamento a que cada lado se compromete ao receber (ou enviar) uma mensagem específica.

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984



# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




A **Camada Física** (L1) é responsável por transferir bits de dados em bruto entre dispositivos através de um meio, o que pode envolver modular sinais de luz (cabos óticos), impulsos elétricos (com fios), ou ondas eletromagnéticas (sem fios).

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




A **Camada de Ligação de Dados** (L2) é responsável por gerir a comunicação entre dois dispositivos “diretamente ligados”. Pode também lidar com erros que ocorram na camada abaixo.

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




A **Camada de Rede** (L3) é responsável por entregar pequenas unidades de dados aos seus destinos corretos, com base num endereço lógico (identificador) para esse destino.

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




A **Camada de Transporte** (L4) é responsável por controlar a transferência de dados entre sistemas e hosts. Gere o ritmo da comunicação, a ordenação dos dados, e realiza verificação de erros de alto nível.

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




A **Camada de Sessão** (L5) controla as interações entre diferentes computadores, isto é, gere a noção de um canal de comunicação entre dois hosts terminais.

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




A **Camada de Apresentação** (L6) transforma a representação dos dados usada pelas aplicações num formato compatível com a camada de sessão, e vice-versa. Também gere aspetos como a cifragem e decifragem de dados (se exigido pela aplicação).

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




A **Camada de Aplicação** (L7) expõe recursos ou serviços de rede a utilizadores finais e aplicações de software.

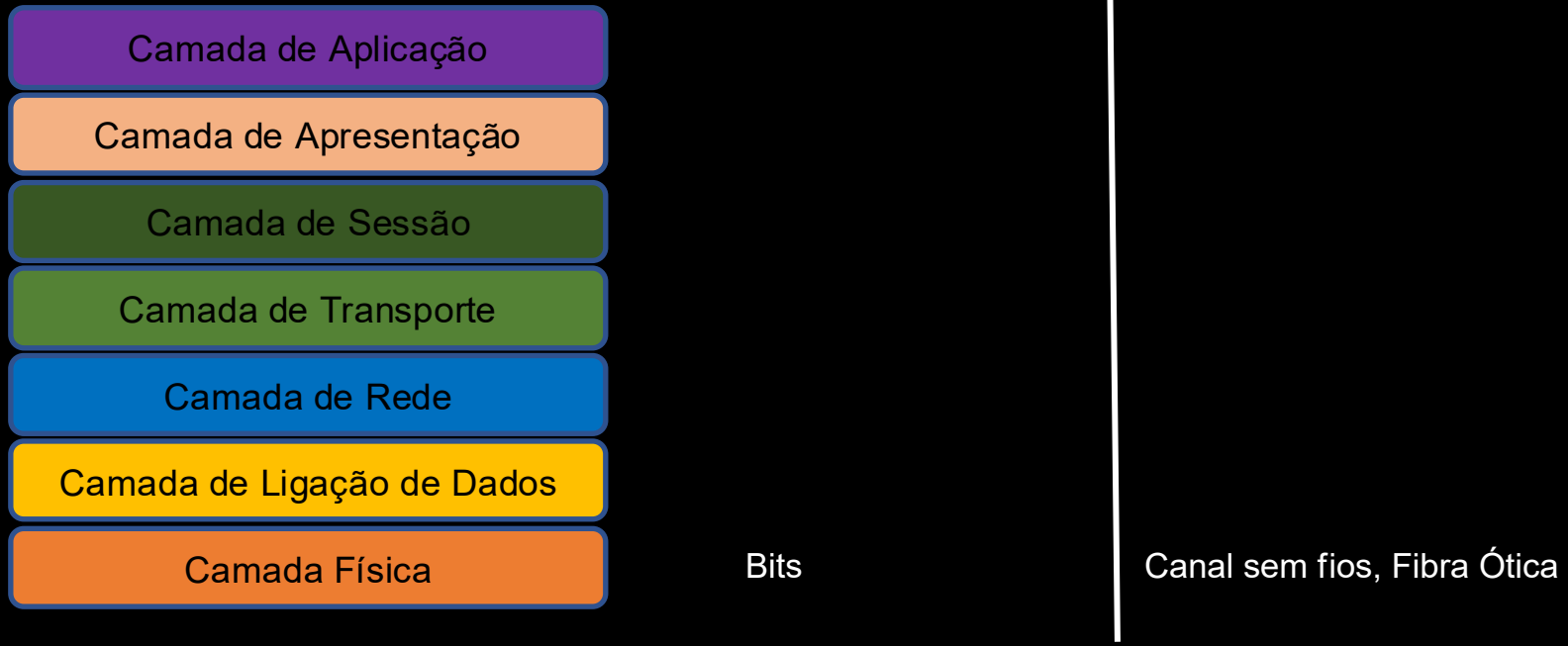
# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984



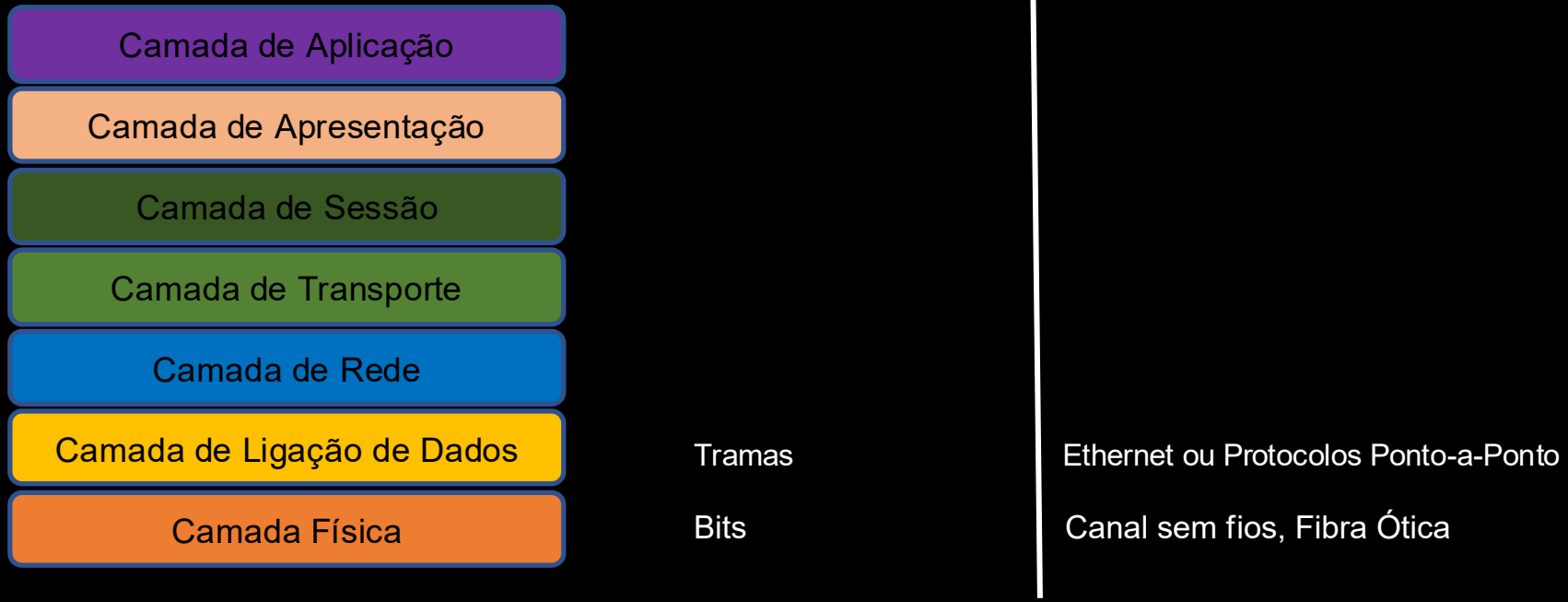
# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984



# Uma pilha de rede abstrata

## Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984



# Uma pilha de rede abstrata

## Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




| | |
|---|---|
| Pacote | IP |
| Tramas | Ethernet ou Protocolos Ponto-a-Ponto |
| Bits | Canal sem fios, Fibra Ótica |

# Uma pilha de rede abstrata

## Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




| | |
|---|---|
| Segmento/Datagrama | TCP, UDP |
| Pacote | IP |
| Tramas | Ethernet ou Protocolos Ponto-a-Ponto |
| Bits | Canal sem fios, Fibra Ótica |

# Uma pilha de rede abstrata

## Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




| | |
|---|---|
| Dados | Socket |
| Segmento/Datagrama | TCP, UDP |
| Pacote | IP |
| Tramas | Ethernet ou Protocolos Ponto-a-Ponto |
| Bits | Canal sem fios, Fibra Ótica |

# Uma pilha de rede abstrata

## Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




| | |
|---|---|
| Dados | TLS, SSH, FTP |
| Dados | Socket |
| Segmento/Datagrama | TCP, UDP |
| Pacote | IP |
| Tramas | Ethernet ou Protocolos Ponto-a-Ponto |
| Bits | Canal sem fios, Fibra Ótica |

# Uma pilha de rede abstrata

## Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




| | |
|---|---|
| Dados | HTTP, FTP |
| Dados | TLS, SSH, FTP |
| Dados | Socket |
| Segmento/Datagrama | TCP, UDP |
| Pacote | IP |
| Tramas | Ethernet ou Protocolos Ponto-a-Ponto |
| Bits | Canal sem fios, Fibra Ótica |

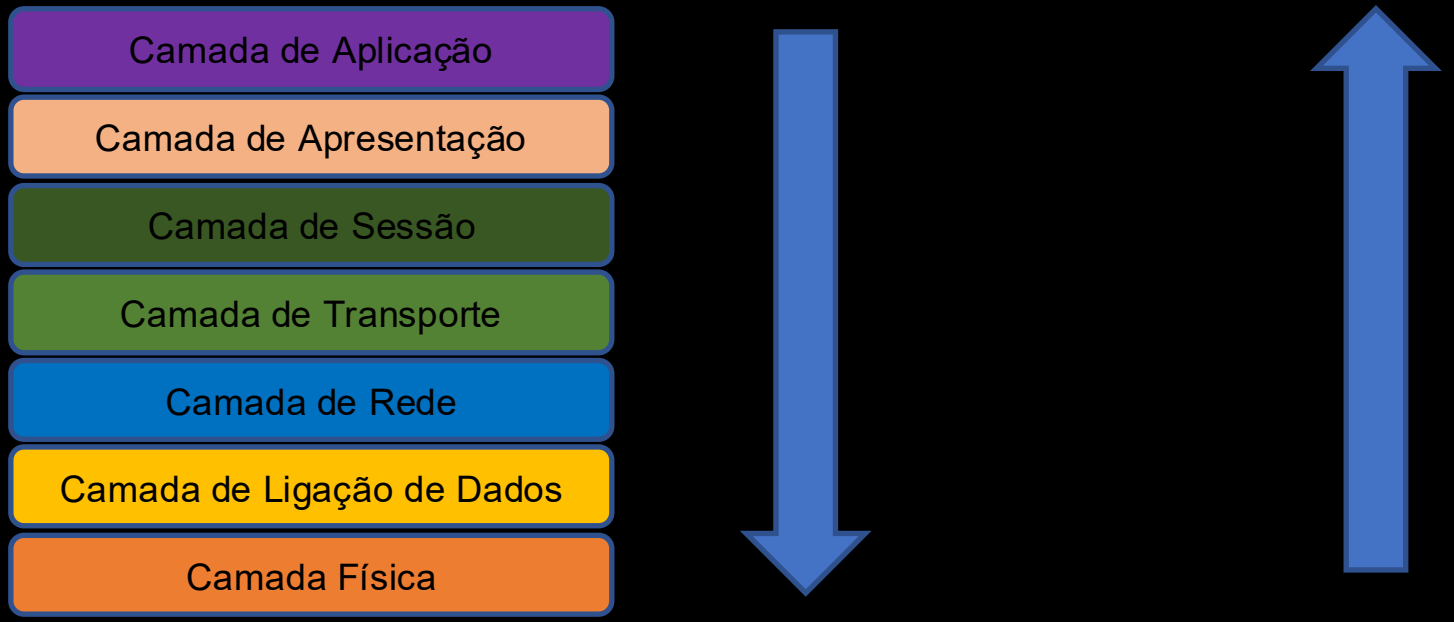
# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984



# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984



# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




A materialização de cada uma destas camadas (abstratas) é um Protocolo.

# Uma pilha de rede abstrata

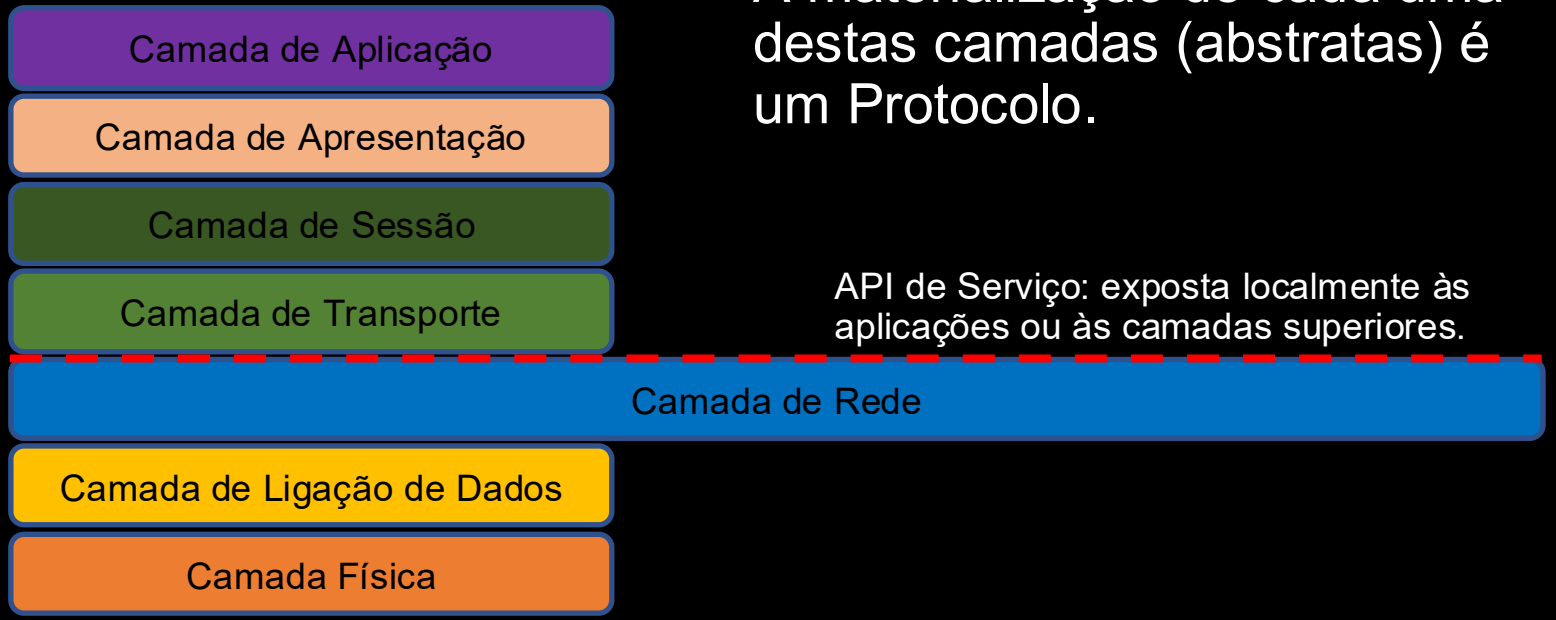
Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




A materialização de cada uma destas camadas (abstratas) é um Protocolo.

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984



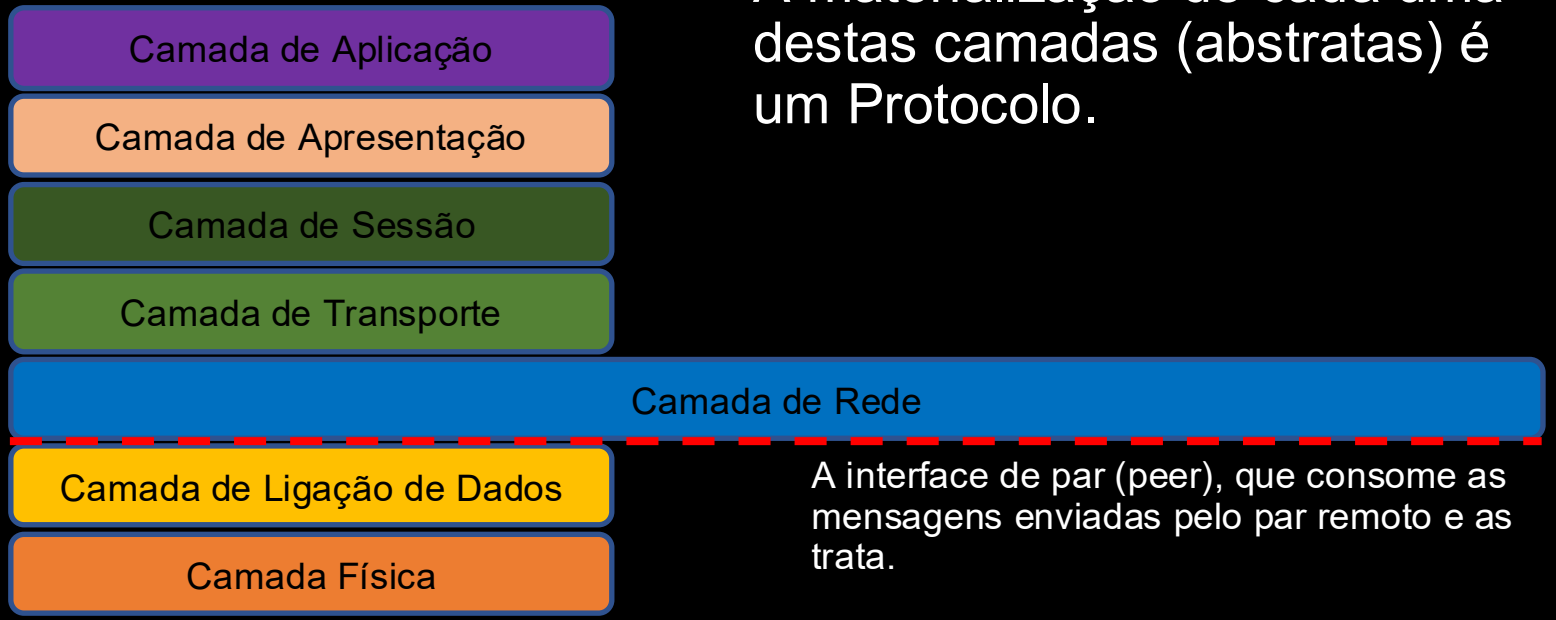

A materialização de cada uma destas camadas (abstratas) é um Protocolo.

API de Serviço: exposta localmente às aplicações ou às camadas superiores.

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984

A materialização de cada uma destas camadas (abstratas) é um Protocolo.




A interface de par (peer), que consome as mensagens enviadas pelo par remoto e as trata.

# Uma pilha de rede abstrata

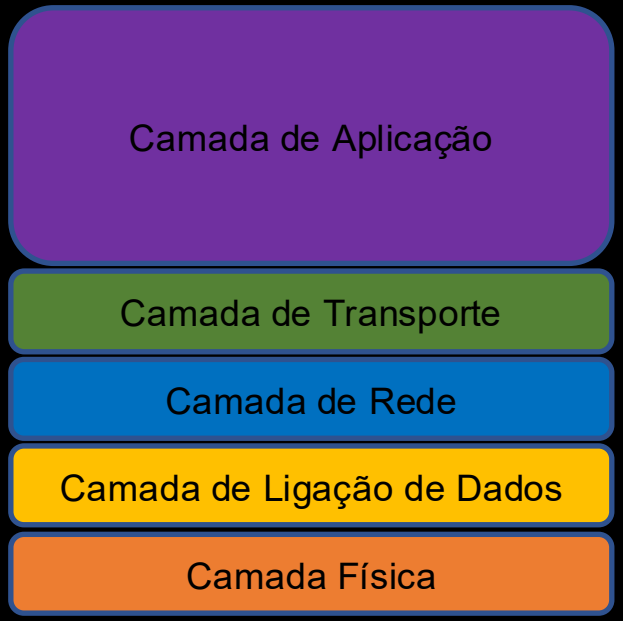
Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




Em muitos casos, a funcionalidade fornecida pelas Camadas de Sessão e Apresentação acaba por ser tratada em conjunto pela Camada de Aplicação.

# Uma pilha de rede abstrata

Modelo de Interligação de Sistemas Abertos (Modelo OSI) - 1984




Em muitos casos, a funcionalidade fornecida pelas Camadas de Sessão e Apresentação acaba por ser tratada em conjunto pela Camada de Aplicação.

# A Heterogeneidade no Modelo OSI

- Em teoria, o modelo OSI permite múltiplas implementações e desenhos para cada uma das suas camadas.

# A Heterogeneidade no Modelo OSI

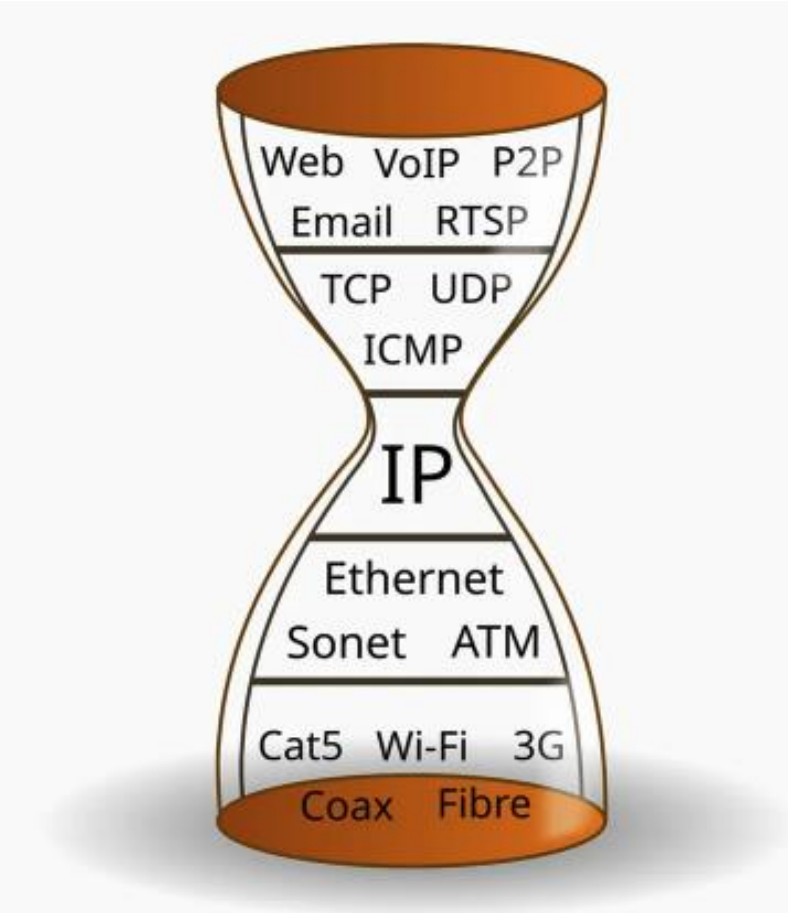
- Em teoria, o modelo OSI permite múltiplas implementações e desenhos para cada uma das suas camadas.
- Até certo ponto isto é verdade:
  - Muitas tecnologias de ligação na base — Wi-Fi, Ethernet, fibra, satélite.
  - Vários protocolos de Transporte — UDP, TCP
  - Múltiplas aplicações no topo — e-mail, streaming, http, …

# A Heterogeneidade no Modelo OSI

- Em teoria, o modelo OSI permite múltiplas implementações e desenhos para cada uma das suas camadas.
- Até certo ponto isto é verdade:
  - Muitas tecnologias de ligação na base — Wi-Fi, Ethernet, fibra, satélite.
  - Vários protocolos de Transporte — UDP, TCP
  - Múltiplas aplicações no topo — e-mail, streaming, www, …
- Mas ao nível da rede, existe um único protocolo:
  - O Protocolo de Internet: IP

# A Heterogeneidade no Modelo OSI

- Em teoria, o
  desenhos pa
- Até certo por
  - Muitas tec
    satélite.
  - Vários prot
  - Múltiplas a
- Mas ao nível
  - O Protocol




plementações e
.
Ethernet, fibra,
g, http, …
olo:

# A Cintura Estreita da Internet

Conclusão: esta cintura estreita na Camada de Rede com o único Protocolo de Internet é exatamente o que torna possível a “rede de redes” da última aula.
Quaisquer duas redes conseguem interoperar desde que ambas concordem em transportar **IP** — independentemente do que cada uma faz acima ou abaixo dele.

# Encapsulação

- Sigam uma mensagem a descer a pilha, do lado de quem envia.

# Encapsulação

- Sigam uma mensagem a descer a pilha, do lado de quem envia.
- Cada camada envolve os dados que recebe da camada acima com o seu próprio cabeçalho — a isto chama-se **encapsulação**.

# Encapsulação

- Sigam uma mensagem a descer a pilha, do lado de quem envia.
- Cada camada envolve os dados que recebe da camada acima com o seu próprio cabeçalho — a isto chama-se **encapsulação**.
- Do lado de quem recebe, ocorre o processo inverso: cada camada remove o seu próprio cabeçalho e entrega o resto acima — **desencapsulação**.

# Encapsulação (Exemplo)




- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

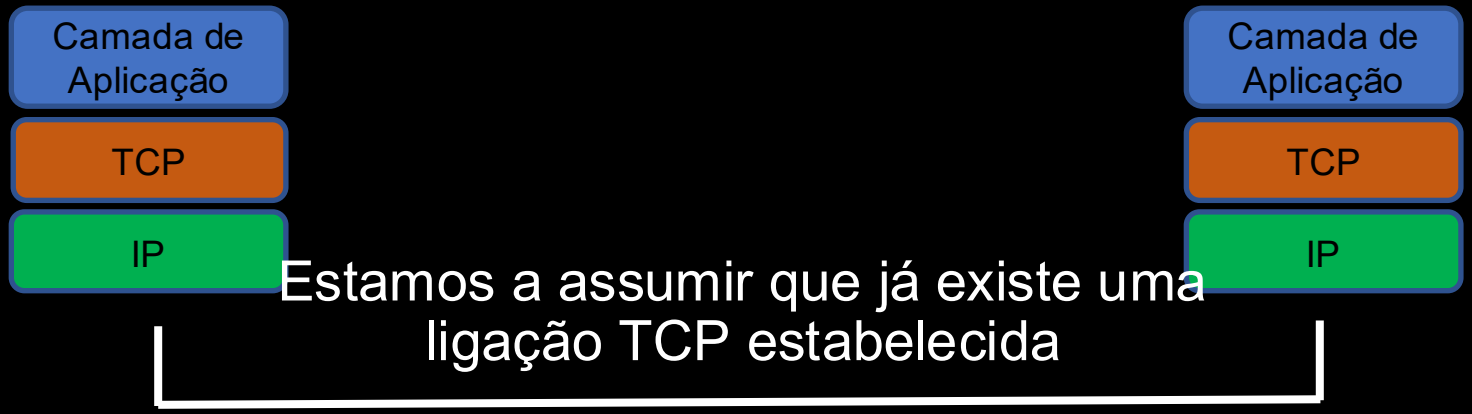
# Encapsulação (Exemplo)

Browser



Servidor HTTP






- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

Browser



O Browser gera um pedido HTTP
(que são dados em texto)

Servidor HTTP






- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

Browser

Pedimos ao TCP para enviar o pedido

Servidor HTTP








- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

Browser



Pedimos ao TCP para enviar o pedido

O TCP encapsula os dados do HTTP REQ num Segmento TCP (acrescentando um cabeçalho)

Servidor HTTP






- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

Browser



Servidor HTTP



O TCP passa o seu segmento ao IP para o entregar à máquina certa.




- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

Browser



Servidor HTTP



O TCP passa o seu segmento ao IP para o entregar à máquina certa.

E o IP encapsula o segmento TCP num pacote IP (com o seu próprio cabeçalho)




- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

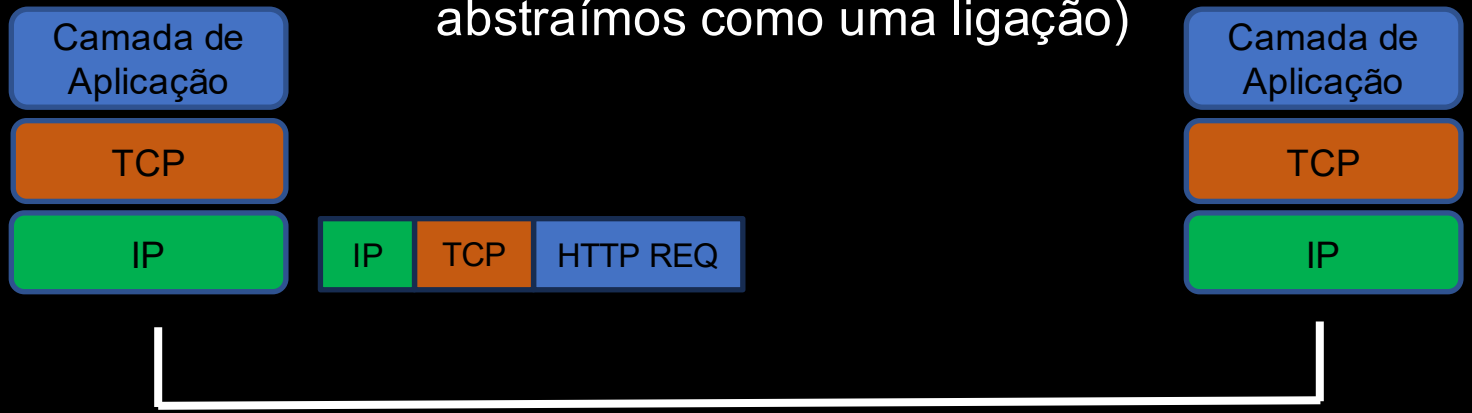
Browser



Agora vamos abstrair alguns dos outros protocolos e assumir que conseguimos encaminhar este pacote IP até ao host de destino, passando por várias redes (que abstraímos como uma ligação)

Servidor HTTP






- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

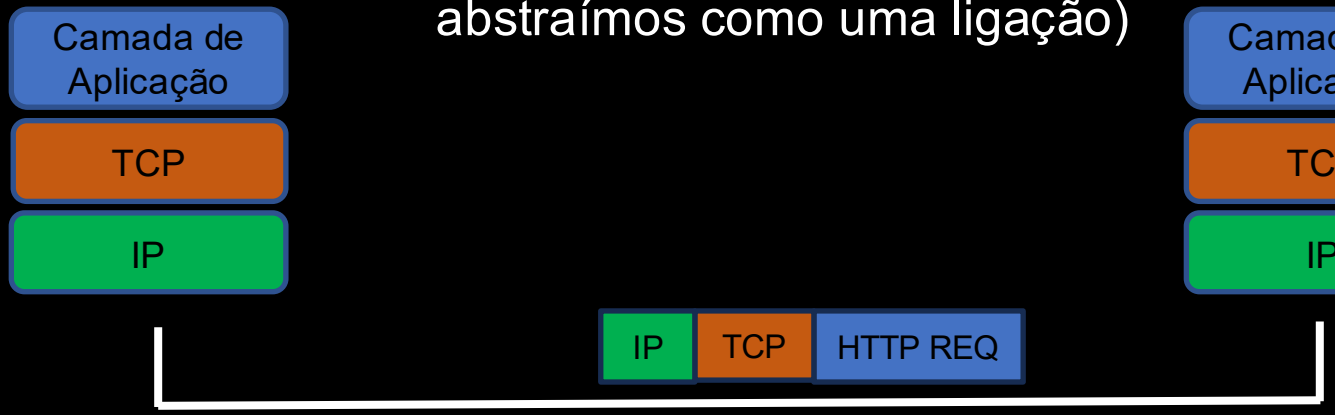
Browser



Agora vamos abstrair alguns dos outros protocolos e assumir que conseguimos encaminhar este pacote IP até ao host de destino, passando por várias redes (que abstraímos como uma ligação)

Servidor HTTP






- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

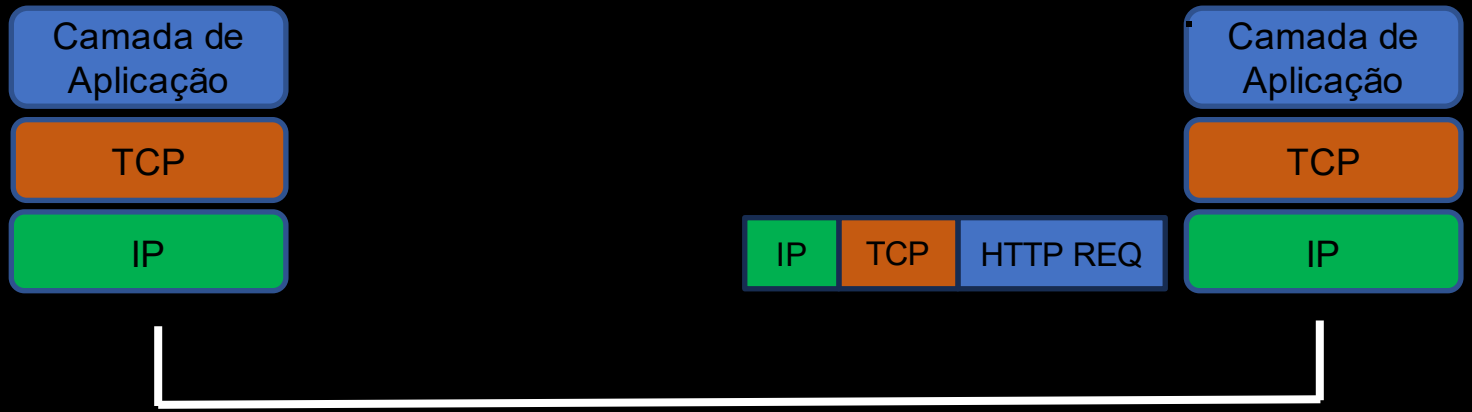
Browser



Servidor HTTP



Ao chegar ao outro host, o protocolo IP no recetor processa o pacote




- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

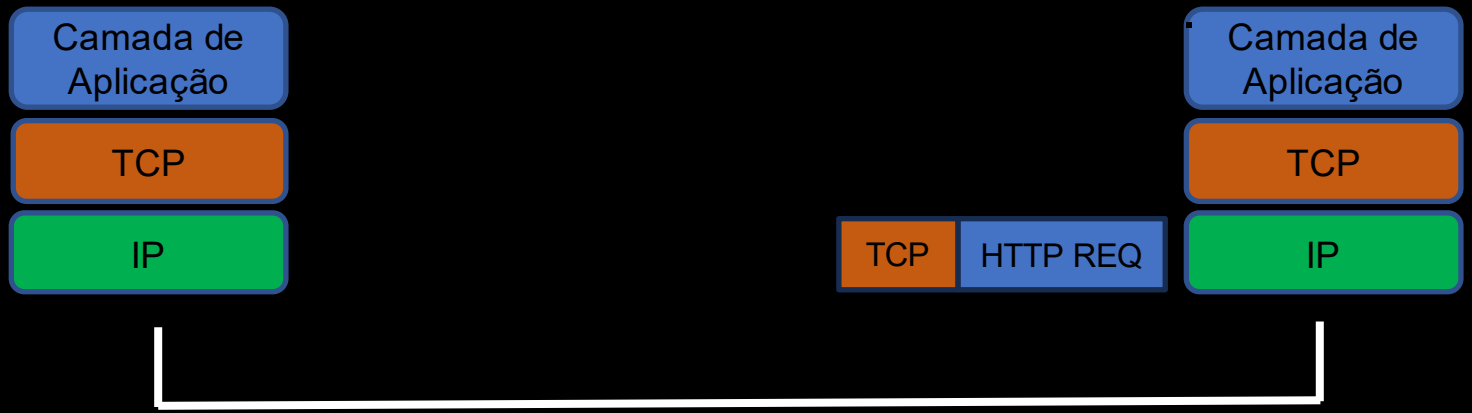
Browser



Ao chegar ao outro host, o protocolo IP no recetor processa o pacote, remove o seu cabeçalho (desencapsula o segmento TCP)

Servidor HTTP






- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

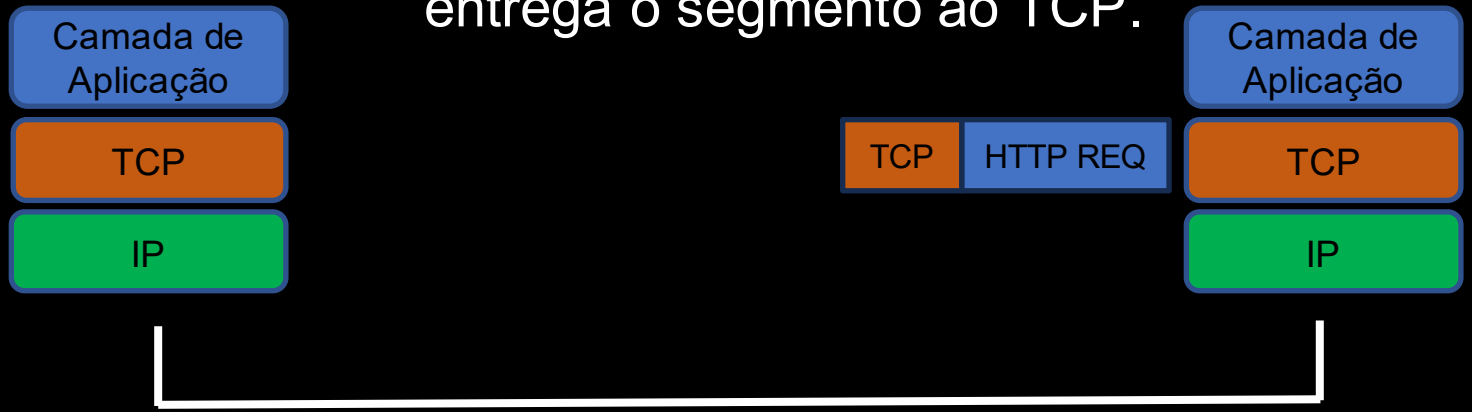
Browser



Servidor HTTP



Ao chegar ao outro host, o protocolo IP no recetor processa o pacote, remove o seu cabeçalho (desencapsula o segmento TCP), e entrega o segmento ao TCP.




- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

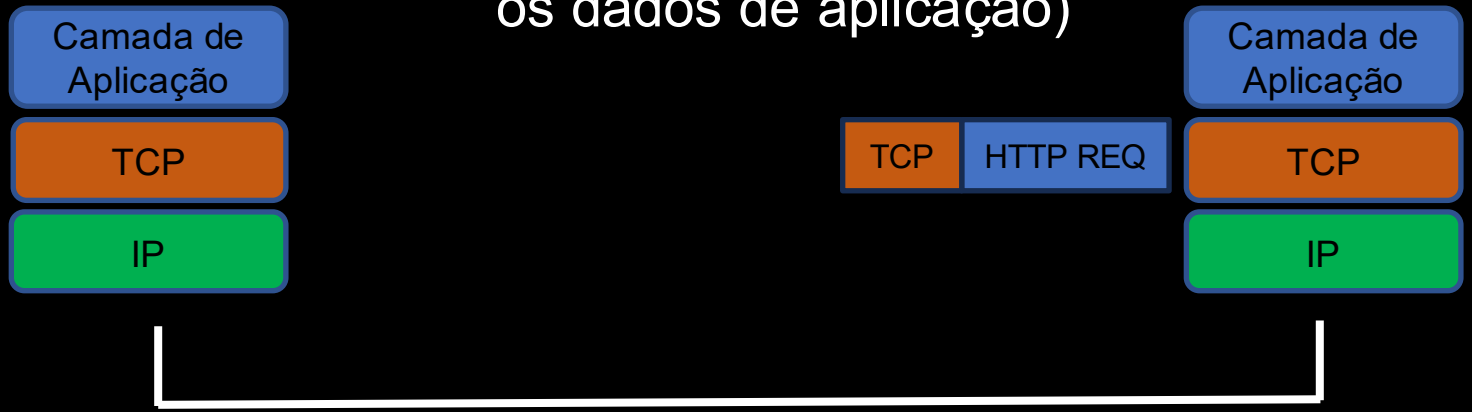
Browser



Servidor HTTP



O TCP faz o mesmo: processa o cabeçalho TCP (que, por exemplo, contém informação que identifica a que processo devem ser entregues os dados de aplicação)




- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

Browser



Desencapsula os dados de aplicação removendo o cabeçalho TCP

Servidor HTTP






- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

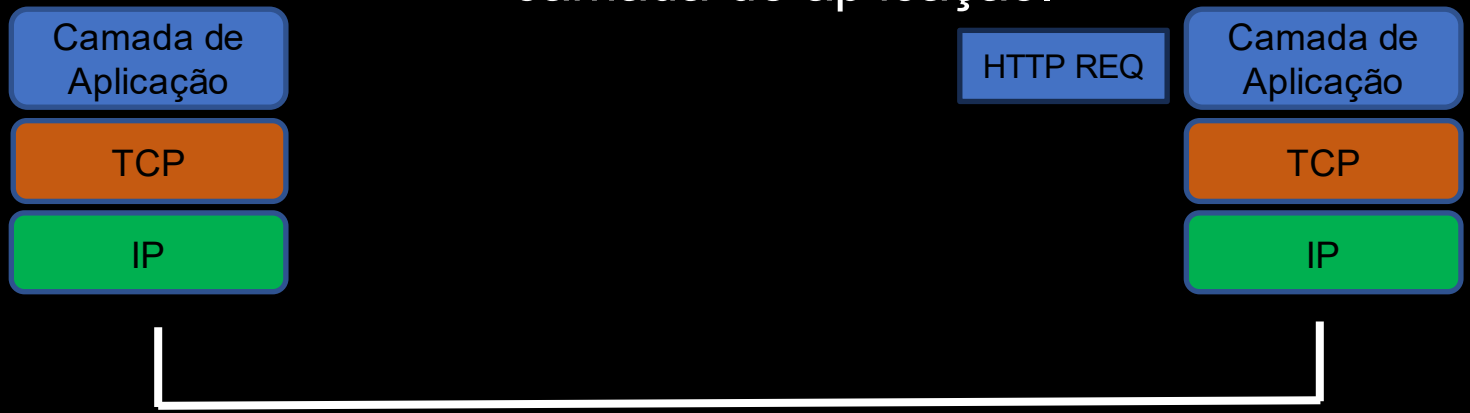
Servidor HTTP

Browser



Desencapsula os dados de aplicação removendo o cabeçalho TCP e entrega os dados da aplicação à camada de aplicação.






- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Encapsulação (Exemplo)

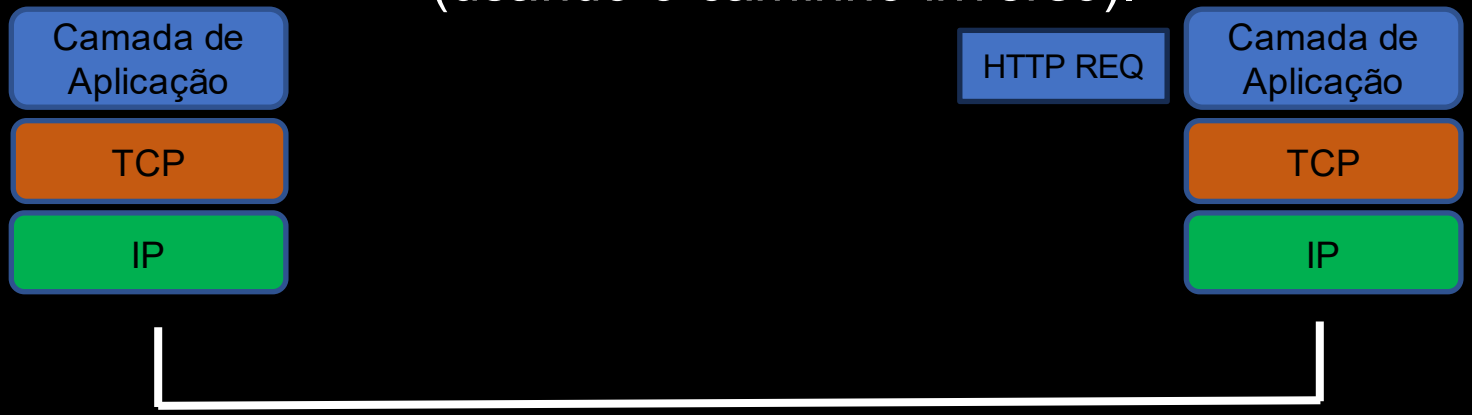
Browser



Finalmente, a aplicação recebe o Pedido HTTP, processa-o, e envia uma resposta de volta ao cliente (usando o caminho inverso).

Servidor HTTP






- Lembram-se do pedido HTTP do exemplo de abertura da última aula? Vamos ver como isto funciona (versão simplificada)…

# Interagir Programaticamente com a Pilha de Rede

- Agora que já temos uma ideia da composição da pilha de rede, e de algumas das responsabilidades de algumas dessas camadas, como é que interagimos com a pilha de rede?

- Por outras palavras, qual é a abstração fornecida pelo sistema operativo, que permite a uma aplicação enviar e receber dados de e para outro host na rede?

# Sockets

- Um socket é a abstração que permite a um processo interagir com a camada de rede — a interface entre o vosso programa e o subsistema de rede do sistema operativo.
- **Pensem numa tomada elétrica**: um contacto padronizado entre um aparelho e a rede elétrica. Não importa que aparelho está de um lado, nem que central elétrica está do outro, desde que ambos respeitem o mesmo contacto.

# Sockets

- Um socket é a abstração que permite a um processo interagir com a camada de rede — a interface entre o vosso programa e o subsistema de rede do sistema operativo.
- **Pensem numa tomada elétrica**: um contacto padronizado entre um aparelho e a rede elétrica. Não importa que aparelho está de um lado, nem que central elétrica está do outro, desde que ambos respeitem o mesmo contacto.

Do ponto de vista do sistema operativo, um socket é tratado como um descritor de ficheiro e existe na tabela de ficheiros abertos de um processo.

Isto também implica que operações como write(), close(), entre outras, podem ser chamadas usando um socket como argumento.

Reparem que um processo, um utilizador, e um host como um todo, têm um número máximo de descritores de ficheiro abertos que podem existir; os sockets (abertos) também contam para este limite.

# Sockets

- Um socket é o ponto onde um processo de aplicação se liga à rede — a interface entre o vosso programa e o subsistema de rede do sistema operativo.
- **Pensem numa tomada elétrica**: um contacto padronizado entre um aparelho e a rede elétrica. Não importa que aparelho está de um lado, nem que central elétrica está do outro, desde que ambos respeitem o mesmo contacto.

```
socket(domain, type, protocol)
```

- **domain** define a família de protocolos (Internet, ou algo completamente diferente, como um socket Unix local); **type** escolhe a semântica de comunicação; **protocol** normalmente é deixado ao critério do sistema, dados os dois primeiros.

# Dois **type** de Socket

- **SOCK_STREAM**
- Um fluxo de bytes fiável
- Aquilo em que o TCP se baseia
- Coberto em detalhe, daqui a duas semanas

- **SOCK_DGRAM**
- Mensagens individuais, sem garantias
- Aquilo em que o UDP se baseia
- Vão usar exatamente isto, na aula prática desta semana

# Dois **type** de Socket

- Hoje só precisam de saber que estas duas opções existem.
- Ainda **não** estamos a resolver a tensão entre elas — vamos discutir isto mais à frente nesta cadeira.

# O Ciclo de Vida de um Socket

- **Servidor — open passivo**
- `socket()`
- `bind()`
- `listen()`
- `accept()`

- **Cliente — open ativo**
- `socket()`
- `connect()`

# O Ciclo de Vida de um Socket

- O servidor é quem **espera que o telefone toque** — listen(), depois accept().
- O cliente é quem **liga** — connect().
- A assimetria importa: o servidor não faz ideia antecipadamente de quem se vai ligar. O cliente sabe exatamente a quem está a ligar.
- Mais uma coisa: listen() e accept() só fazem sentido para SOCK_STREAM. Com SOCK_DGRAM não há nenhuma “ligação” — cada mensagem já traz o endereço do remetente. Vão ver exatamente isso, esta semana, na aula prática.

# Multiplexagem de Aplicações

- Imaginem o vosso próprio portátil com: um browser aberto, e um cliente de email aberto, ambos a falar com a rede ao mesmo tempo.

# Multiplexagem de Aplicações

- Imaginem o vosso próprio portátil com: um browser aberto, e um cliente de email aberto, ambos a falar com a rede ao mesmo tempo.
- Chega um único segmento, através da vossa (única) ligação de rede.

# Multiplexagem de Aplicações

- Imaginem o vosso próprio portátil com: um browser aberto, e um cliente de email aberto, ambos a falar com a rede ao mesmo tempo.
- Chega um único segmento, através da vossa (única) ligação de rede.
- Como sabe o sistema operativo a qual das duas aplicações o datagrama é realmente destinado?

# Multiplexagem de Aplicações

- Imaginem o vosso próprio portátil com: um browser aberto, e um cliente de email aberto, ambos a falar com a rede ao mesmo tempo.
- Chega um único segmento, através da vossa (única) ligação de rede.
- Como sabe o sistema operativo a qual das duas aplicações o datagrama é realmente destinado?
- Já agora — isto não é hipotético. Acontece em todos os dispositivos, a cada segundo.

# Multiplexagem de Aplicações

- Imaginem o vosso próprio portátil com: um browser aberto, e um cliente de email aberto, ambos a falar com a rede ao mesmo tempo.
- Chega um único segmento, através da vossa (única) ligação de rede.
- Co ões o da
- Já di

Podem pensar nisto como uma pergunta mais geral. Dentro de uma máquina, consigo identificar um processo pelo seu identificador de processo, mas esse número só faz sentido dentro dessa máquina.

Numa perspetiva de redes de computadores, como consigo identificar um processo específico que está a correr num host específico (provavelmente consigo identificar o host usando o seu endereço IP)?

# Portas

- **Uma porta**: o identificador que resolve exatamente esta ambiguidade — uma “chave de desmultiplexagem” (demux key), usada para desmultiplexar mensagens recebidas para o processo certo.
- Pensem num prédio de apartamentos: o endereço IP — a cintura estreita de há pouco — leva a mensagem ao prédio certo. A porta leva-a ao apartamento certo lá dentro.

# Portas e bind()

- Um socket não tem porta até lhe darem uma.
- **bind()** é o momento em que isso acontece: é o que efetivamente reserva uma porta específica para este socket.
- A partir desse momento, o sistema operativo encaminha tudo o que chega a essa porta diretamente para este socket.
- Em geral, apenas um processo pode fazer bind a um dado par (endereço IP, porta) de cada vez.
- Tentem fazer bind() a uma porta que outra pessoa já está a usar, e recebem um erro — “Address already in use” (EADDRINUSE).
- O cliente normalmente dispensa completamente o bind(): o sistema operativo escolhe-lhe automaticamente uma porta livre (efémera), para que muitos clientes possam executar ao mesmo tempo sem nunca colidirem.

# Juntando Tudo (UDP)

**Servidor:**

```
socket()
bind()
for (;;) {
    recvfrom()
    sendto()
}
close()
```

**Cliente:**

```
socket()
sendto()
recvfrom()
close()
```

# sendto() e recvfrom(), no Ciclo

- Reparem no que falta: nem listen(), nem accept() no servidor — e também nenhum connect() no cliente.
- O UDP não tem noção de “ligação” — por isso não há nada para esperar (listen) ou aceitar (accept), nem nada a que se ligar (connect) antecipadamente.
- **recvfrom()**: lê um datagrama E diz-vos quem o enviou — não sabem quem vos vai escrever até uma mensagem realmente chegar.
- **sendto()**: cada chamada pode ter um destino diferente — nada vos mantém “ligados” a um par específico.
- É exatamente por isso que o ciclo do servidor parece diferente do de um servidor TCP: em vez de fazer accept() a um cliente e depois fazer read()/write() com esse cliente, faz recvfrom()/sendto() com quem quer que apareça, um datagrama de cada vez.

# Juntando Tudo (UDP)

- Reparem: não indiquei quais são os argumentos destas APIs — por exemplo, não está aqui preenchido nenhum campo de endereço.
- Isso é deliberado — fica para a aula prática, onde vão ver a struct sockaddr_in preenchida, campo a campo, em C real.

- A aula prática também vai explicar porque existem funções como htons(), e porque têm de as usar.
- Também vos vai apresentar a visão geral do vosso primeiro projeto (que vamos começar para a semana).

# Resumo

- Porque é que as redes são divididas em camadas, e o que é a cintura estreita da Internet.
- O significado formal de host/nó, ligação e protocolo.
- O que é a encapsulação.
- O que é um socket, e a distinção entre open passivo / open ativo.
- O que é uma porta, e porque se chama uma demux key.
- Para a próxima aula: UDP, em detalhe — e esta semana, na aula prática, tudo isto torna-se código C real.

# Bibliografia


- Peterson, L. L., & Davie, B. S. (2019). *Computer Networks: A Systems Approach* (Release Version 6.1). Chapter 1, Sections 1.3-1.4.

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

©2026 NOVA FCT